

CanlıFal Flutter — Canlı Falcılar, TRTC, Hediye & Koltuk Sistemi Prompt'u

Bu prompt, backend'de yapılan güncellemelerin (11 koltuk, otomatik oturma, VIP/şifreli oda kontrolü, hediye render meta verisi) Flutter uygulamasında birebir çalışması için gereken tüm değişiklikleri içerir. Tüm iş/finans mantığı backend'dedir; Flutter yalnızca backend'in gönderdiği veriyi **render eder**.

```
Base URL      : https://canlifal.com
Auth          : Authorization: Bearer <jwt_token> (POST /api/auth/mobile-login)
Ses/Video     : Tencent TRTC (SDK App ID: 20040423)
Flutter SDK   : trtc_sdk (v5) → paket: trtc_sdk
UserSig       : POST /api/trtc/token
Gerçek zaman  : SSE → GET /api/chat/rooms/{roomId}/stream
```

0. GENEL KURAL: SADECE TENCENT TRTC — AGORA VE DİĞERLERİ KALDIRILACAK

Uygulamada ses/görüntü için **yalnızca Tencent TRTC** kullanılacak. Agora ve diğer tüm RTC/ses SDK'ları tamamen kaldırılacak.

0.1 pubspec.yaml temizliği

Aşağıdaki paketleri **SİL** (varsa):

```
# KALDIRILACAK — hiçbiri kalmayacak
agora_rtc_engine: ...
agora_rtc_engine_example: ...
agora_rtm: ...
zego_express_engine: ...
zego_uikit: ...
flutter_webrtc: ... # TRTC dışında bağımsız WebRTC kullanılmayacak
jitsi_meet_flutter_sdk: ...
twilio_programmable_video: ...
```

Sadece şu kalacak:

```
dependencies:
  trtc_sdk: ^5.x.x # Tencent TRTC v5 — tek ses/görüntü motoru
```

0.2 Kod temizliği (zorunlu)

- `import 'package:agora_rtc_engine/...'` içeren **tüm** dosyaları bul ve kaldır.
- `RtcEngine`, `AgoraVideoView`, `ChannelProfileType`, `ClientRoleType`, `createAgoraRtcEngine()` vb. Agora sembollerini içeren tüm çağrılarını TRTC karşılıklarıyla değiştir.
- `android/app/src/main/AndroidManifest.xml`, `ios/Runner/Info.plist` içindeki Agora'ya özel izin/anahtar satırlarını kaldır. TRTC için gereken mikrofon/kamera izinleri kalacak.

- Grep kontrolü (kod tabanında hiç sonuç dönmemeli):

```
bash
grep -rin "agora\|zego\|jitsi\|twilio.*video" lib/
```

0.3 TRTC bağlantı akışı (özet)

```
// 1) UserSig al
final res = await api.post('/api/trtc/token', body: { 'roomId': roomId, 'role': is-
Speaker ? 'host' : 'audience' });
final d = res['data'];
final int sdkAppId = d['sdkAppId'];
final String userId = d['userId'];
final String userSig = d['userSig'];
final String trtcRoomId = d['trtcRoomId']; // 'voice_room_<chatRoomId>' - web ile
AYNI olmalı
final int numericUid = d['numericUid'];

// 2) TRTC odasına gir (v5)
final trtc = (await TRTCCloud.sharedInstance())!;
trtc.enterRoom(TRTCParams(
  sdkAppId: sdkAppId,
  userId: userId,
  userSig: userSig,
  strRoomId:
trtcRoomId, // strRoomId kullan (string oda) - numeric roomId DEĞİL
  role: isSpeaker ? TRTCRoleType.anchor : TRTCRoleType.audience,
), TRTCAppScene.voiceChatRoom);
```

⚠ Web ve Flutter **aynı trtcRoomId string'ini** kullanmak zorunda, aksi halde aynı odada birbirlerini duyamazlar. Bu değeri **kendin üretme**, her zaman `/api/trtc/token` yanıtındaki `trtcRoomId` 'yi kullan.

1. SESLİ ODA: 11 KOLTUK + OTOMATİK OTURMA

Backend artık **11 koltuk** (index 0-10) döndürüyor. Flutter tarafında sabit `15` varsa **11** yapılacak; en doğrusu backend'in döndürdüğü liste uzunluğuna göre çizmek.

1.1 Koltuk sayısı

```
const int kSeatCount = 11; // 0..10
```

- Koltuk gridini `state.seats.length` (backend 11 dolu/boş slot döner) üzerinden çiz; sabit sayıya güvenme.
- GET `/api/chat/rooms/{roomId}/state` → `seats` : 11 elemanlı dizi. Her eleman `null` (boş) ya da `{ seatIndex, userId, name, nickname, image, micOn, receivedJetons }`.

1.2 Otomatik oturma (backend hallediyor)

Backend, odaya **ilk kez** giren yetkili kullanıcıyı boş ilk koltuğa (0→10) otomatik oturtur. Flutter'ın yapması gerekenler:

- Odaya girişte **koltuk seçmeden** POST `/api/live/join-room` (veya POST `/api/chat/rooms/{roomId}/presence`) çağır; backend `seatIndex` 'i otomatik atar ve yanıtta/ `state` 'te döner.
- Giriş yanıtındaki (veya hemen ardından çekilen `state` 'teki) kendi `seatIndex` 'ini oku ve mikrofona

rolünü ona göre ayarla (`seatIndex >= 0` ise `anchor` , değilse `audience`).

- **Tüm koltuklar doluysa** kullanıcı dinleyici (`seatIndex = -1`) kalır — bu normal, hata gösterme.
- Heartbeat/yeniden bağlanmada koltuk **korunur**; her heartbeat'te `seatIndex` göndermeye devam et (mevcut değeri), böylece koltuğun düşmez.

1.3 Koltuk işlemleri (elle)

```
POST /api/live/seats
body: { roomId, action: 'take'|'leave'|'swap'|'force', seatIndex?, targetUserId? }
- take : kendi koltuğunu al (0..10)
- leave : koltuğu boşalt (seatIndex = -1)
- swap : (yetkili) targetUserId'yi seatIndex'e taşı
- force : (yetkili) targetUserId'yi koltuktan indir
```

Geçersiz koltukta backend `INVALID_SEAT` (0-10) döner; UI'da koltuk numaralarını 0-10 ile sınırla.

2. VIP / ŞİFRELİ ODA GİRİŞİ

Backend artık **VIP** ve **NORMAL** tipteki odalarda, oda sahibinin belirlediği şifreyi giriş anında doğruluyor. Oda sahibi ve yetkili (site admin/moderatör veya oda içi `sop` ve üzeri) şifresiz girer.

2.1 Şifre sorma akışı

1. Odaya girmeden önce oda tipini kontrol et: `roomAccessType` / `roomType` alanı `VIP` veya `NORMAL` ve odada şifre tanımlıysa kullanıcıdan şifre iste.

2. `POST /api/live/join-room` (veya `presence`) çağrısına şifreyi ekle:

```
json
{ "roomId": "...", "roomType": "voice", "password": "<kullanıcının girdiği şifre>" }
```

3. Backend şifre hatalıysa **403** döner:

```
json
{ "success": false, "error": { "code": "INVALID_ROOM_PASSWORD", "message": "Oda şifresi hatalı" } }
```

Bu durumda kullanıcıya “Oda şifresi hatalı” göster ve tekrar şifre iste; **odaya girme, TRTC’ye bağlanma**.

4. Şifre doğruysa normal akışla devam (TRTC token → `enterRoom`).

2.2 VIP oda oluştururken şifre koyma (oda sahibi)

VIP oda açan kullanıcı için oda ayarlarında “Oda Şifresi” alanı olacak. Şifre backend’de `Chat-Room.password` alanında saklanır (mevcut oda ayarları/oluşturma endpoint’i ile). Şifre boş bırakılırsa oda şifresiz olur.

Not: Şifre yalnızca sunucuda doğrulanır; Flutter şifreyi asla lokalde kontrol etmez, her zaman backend’e gönderir.

3. HEDİYE RENDER SİSTEMİ (HERKESTE AYNI GÖRÜNSÜN)

Backend, her hediye olayında (SSE + HTTP yanıtı) artık **render meta verisi** gönderiyor. Böylece hediyeye **her telefonda birebir aynı** görünür ve odadaki/yayındaki **herkes** görür.

3.1 Hediye olayının geldiği yerler

- **SSE (canlı):** GET /api/chat/rooms/{roomId}/stream → type: "gift" olayları. Aşağıdaki alanlar olayın içindedir.
- **Sesli oda hediye gönder:** POST /api/chat/rooms/{roomId}/gifts body { giftTypeId, quantity } → yanıtta gift + render alanları.
- **Birleşik gönderim:** POST /api/live/gift/send body { roomId, roomType:'voice'|'stream', giftTypeId, quantity }.
- **Canlı yayın hediye:** POST /api/video-streams/{streamId}/gifts → yanıtta giftRender ; SSE type:'gift' olayında render alanları.
- **Geç bağlananlar / senkron:** GET /api/chat/rooms/{roomId}/gifts → recentGifts (son 15sn) render alanlarıyla döner.

3.2 Render meta alanları (hepsi olayda/yanıtta gelir)

Alan	Tip	Anlamı
giftId	string	Olayın tekil kimliği — animasyonu tekilliştirmek (dedupe) için
timestamp	int	Olayın backend zaman damgası (ms). Katılımdan önceki hediyeleri elemek için
senderId	string	Gönderen kullanıcı id
recipientId / receiverId	string?	Alıcı kullanıcı id (varsa)
giftIcon	string	Küçük ikon/emoji
assetUrl	string?	Oynatılacak asset (resim/video/lottie/svg) URL'i
assetType	string?	image video lottie svg gif
assetFormat	string?	Kesin format (backend türetir): png jpeg webp avif gif svg lottie mp4 webm . Oynatıcı seçimini buna göre yap
imageUrl	string?	Resim/animasyon asset URL'i (png/gif/webp/lottie/svg)
videoUrl	string?	Yalnızca mp4 / webm için video URL'i
thumbnailUrl	string?	Küçük önizleme/placeholder (precache + video poster)
animationType	string?	Animasyon tipi/isim (gift-Type.animation ?? display-Type)
animationDurationMs	int?	Animasyon süresi (ms)
startDelayMs	int?	Başlangıç gecikmesi (ms)
effectColor	string?	Efekt/parıltı rengi (opsiyonel)
musicUrl	string?	Hediye müziği (opsiyonel)

Alan	Tip	Anlamı
displayType	string?	static animation video fullscreen mini continuous play_once ...
isFullscreen	bool	true → kenarları tam dolduran tam ekran gösterim
visibleAsFullscreen	bool	Tam ekran gösterime uygun mu
visibleInVoiceRoom	bool	Sesli odada gösterilsin mi
visibleInLiveStream	bool	Canlı yayında gösterilsin mi
screenPosition	string?	bottom top center above_seat room_center fullscreen message_area ...
displayDurationMs	int?	Gösterim süresi (ms). Yoksa varsayılan kullan
tier	string?	small big huge
repeatCount	int?	Tekrar sayısı
particleEffect	string?	Partikül efekt adı (opsiyonel)
soundUrl	string?	Efekt sesi (opsiyonel)

3.3 Görüntüleme kuralları (KESİN)

A) Tam ekran (kenarları tam doldursun) — büyük/videolu hediyeler

Koşul: `isFullscreen == true` **veya** `displayType == 'fullscreen'` **veya** `screenPosition == 'fullscreen'` **veya** `tier == 'huge'`.

- Overlay **tüm ekranı** kaplar: `Positioned.fill + BoxFit.cover` (kenarlarda boşluk kalmayacak, tam dolduracak).
- `assetType` :
- `video` → tam ekran video player (`BoxFit.cover`), sesli ise `soundUrl` /videonun kendi sesi.
- `lottie / svg` → tam ekran, `BoxFit.cover` .
- `image / gif` → tam ekran, `BoxFit.cover` .
- Süre: `displayDurationMs` (yoksa video/animasyon süresi; sabit fallback ör. 3500ms).
- Bu hem **canlı yayında** hem **sesli sohbet odalarında** geçerli.

B) Sesli odada büyük ama tam ekran olmayan hediye — koltuk altından mesaj kutusuna kadar

Koşul: sesli odada, `tier big / huge` ya da `assetType == 'video'` ama (A) tetiklenmediyse; `screenPosition above_seat / room_center` gibi.

- Overlay, **koltukların hemen altından başlayıp mesaj yazma kutusunun hemen üstüne kadar** olan orta alanı kaplar (o dikey aralığı büyükçe doldurur).
- Ölçü referansı: üst sınır = koltuk gridinin alt kenarı; alt sınır = mesaj input kutusunun üst kenarı. Bu aralığa yerleştir, `BoxFit.contain / cover` ile ortalama.
- Süre: `displayDurationMs` (yoksa `~3000ms`).

C) Küçük hediye

`tier == 'small' / displayType mini / static` : küçük bir animasyon/ikon olarak `screenPosition` 'a göre (ör. `bottom`) kısa süre göster.

`assetUrl` boşsa `giftIcon` (emoji) ile küçük bir animasyon göster (fallback). Asla boş ekran bırakma.

3.4 “Herkes görsün” ve “her telefonda ayrı ayrı say”

- Odaya/yayına giren **her** istemci SSE'ye abone olur ve gelen `type: 'gift'` olayını **kendi cihazında** render eder → hediye herkeste görünür.
- Her istemci, olaydaki `senderId + quantity` ile **gönderen başına** yerel bir sayaç tutar (ör. `Map<senderId, int>`), böylece “kim kaç hediye attı” her telefonda ayrı ayrı görünür.
- Bu sayaç yalnızca görüntülemedir; toplam/finans backend'de. Liderlik tablosu: `GET /api/chat/rooms/{roomId}/gifts` → `leaderboard`.

4. SESLİ ODA: HEDİYE GÖNDERENLER PANELİ (KOLTUĞUN SOL ALTINDA)

Sesli sohbet odalarında, atılan hediyeleri gönderenlerin isimleri **koltuk alanının sol altında** küçük bir panelde gösterilecek.

4.1 Kurallar (KESİN)

- Konum: **koltukların sol alt köşesi** (koltuk grid alanının sol-alt tarafı).
- En fazla **3 kişi** aynı anda listelenir (en son hediye atan 3 kişi, en yenisi üstte).
- Her isim **4 saniye** görünür; süre dolunca **kararak (fade/darken)** kapanır.
- Burada yalnızca **hediye atan kişinin ismi** görünür (ekstra bilgi/ikon minimal). Gönderenlerin ismi **sadece bu panelde** gösterilir — koltuk ismi veya başka yerde “hediye atan” etiketi olarak tekrar gösterilmez.
- Yeni hediye gelince listeye eklenir; liste 3'ü aşarsa en eski düşer.

4.2 Uygulama (öneri)

```
class GiftSenderTag { final String senderId; final String name; final DateTime shown-
At; ... }

// SSE 'gift' geldiğinde:
void onGift(GiftEvent e) {
  _recentSenders.insert(0, GiftSenderTag(senderId: e.senderId, name: e.senderName, sho
wnAt: DateTime.now()));
  if (_recentSenders.length > 3) _recentSenders.removeLast();
  setState(() {});
  // 4 sn sonra bu etiketi kararima animasyonu ile kaldır
  Future.delayed(const Duration(seconds: 4), () {
    _recentSenders.removeWhere((t) => t.senderId == e.senderId && t.shownAt == e.shown
At);
    if (mounted) setState(() {});
  });
}
```

- UI: Positioned(left: 8, bottom: <koltuk alanının altı>) → Column (max 3), her etiket Anim-atedOpacity ile 4sn sonra opacity: 0 'a (kararima) geçer.
- Etiket içeriği: sadece isim (ör. "Ayşe"). İstenirse arkasında hafif koyu yarı saydam kapsül.

5. GERÇEK ZAMANLI OLAYLAR (SSE) — TAM LİSTE

GET /api/chat/rooms/{roomId}/stream (JWT ile). Gelen data: JSON olayları:

type	İçerik	Flutter aksiyonu
connected	{roomId}	Bağlantı kuruldu
messages	yeni sohbet mesajları	Sohbet listesine ekle
system	sistem mesajları	Sistem baloncuğu
gift	hediye + render meta (bkz. §3)	Hediye animasyonu + gönderen paneli + sayaç
pk	PK skor güncellemesi	PK barını güncelle
room_event	{ kind: user_joined\ user_left\ mic_changed\ seat_changed\ room_closed\ owner_changed, ... }	Koltuk/katılımcı/mic durumunu güncelle
presence	aktif kullanıcı listesi	Katılımcı sayısı/listesi
typing	yazıyor bilgisi	"yazıyor..."
dj / müzik	DJ/müzik durumu	Müzik paneli

- room_event → seat_changed : koltuk haritasını state 'ten tazele veya olaydaki bilgiyle güncelle.
- Reconnect: bağlantı koparsa exponential backoff ile yeniden bağlan; yeniden bağlanınca GET .../gifts?after=<sonGörülenZaman> ve GET .../state ile kaçırılanları tazele.

6. BİLDİRİMLER: TIKLAYINCA HEMEN AÇILSIN (BEKLETMESİN)

Backend bildirimleri OneSignal push + DB kaydı olarak gönderir ve **derin bağlantı** verisini taşır.

6.1 Backend'in gönderdiği veri

- Push (OneSignal) additionalData : { type, targetPath, targetId } .
- DB bildirimi (GET /api/notifications): { id, type, title, message, data, isRead, createdAt }
— data bir JSON string'tir ve roomId / streamId / giftName gibi alanları içerir.

6.2 Flutter kuralları (anında açılış)

1. **Bildirime tıklanınca beklemeden** hedef ekrana geç. Ağ isteği tamamlanmasını bekleyip sonra açma — **önce ekranı aç**, veriyi ekran içinde yükle (skeleton/loader göster).
2. OneSignal OSNotificationClickEvent (veya setNotificationOpenedHandler) içinde:

```
dart
final data = event.notification.additionalData; // { type, targetPath, targetId }
_handleDeepLink(type: data?['type'], targetPath: data?['targetPath'], targetId: data?['targetId']);
```

3. `_handleDeepLink` haritası (tür → ekran):
 - `stream_gift` / `call_incoming` / oda daveti → ilgili odaya/yayına **hemen** `Navigator.push`.
 - `targetPath` doluysa doğrudan onu route olarak kullan; yoksa `type` + `targetId` ile eşle.
 4. **Uygulama kapalıyken (cold start)** gelen tıklama: `getInitialNotification` / OneSignal cold-start verisini uygulama açılır açılmaz işleyip ilgili ekrana yönlendir; splash'ta bekletme.
 5. Odaya girişte oda verisini `POST /api/live/join-room` ile **tek çağrıda** al (bu endpoint room state + TRTC + koltuk + katılımcıları birlikte döner) → böylece ekran anında dolar, çok sayıda ardışık istek beklenmez.
-

7. KABUL KRİTERLERİ (TEST)

- [] Kod tabanında Agora/Zego/Jitsi/Twilio-video importu **yok**; ses/görüntü yalnızca TRTC ile çalışıyor.
 - [] Sesli odada koltuk sayısı **11** (0-10); backend `state.seats` uzunluğuna göre çiziliyor.
 - [] Yetkili kullanıcı odaya girince **otomatik** boş koltuğa oturuyor; koltuk doluysa dinleyici kalıyor; heartbeat'te koltuk düşmüyor.
 - [] VIP/şifreli odaya yanlış şifreyle girişte "Oda şifresi hatalı" gösteriliyor ve TRTC'ye bağlanılmıyor; doğru şifre/oda sahibi/yetkili sorunsuz giriyor.
 - [] Büyük/videolu hediyeler **kenarları tam dolduran** tam ekran gösteriliyor (hem sesli oda hem canlı yayın).
 - [] Sesli odada büyük hediye, **koltuk altından mesaj kutusunun üstüne kadar** olan alanı kaplıyor.
 - [] Atılan hediye **odadaki/yayındaki herkeste** görünüyor; her cihaz gönderen başına sayacı ayrı ayrı gösteriyor.
 - [] Koltuğun **sol altında** son 3 hediye göndereninin ismi **4'er saniye** görünüp **kararak** kapanıyor; isimler sadece bu panelde.
 - [] Bildirime tıklanınca ilgili ekran **anında** açılıyor (önce ekran, sonra veri); cold start dahil.
-

8. BACKEND ENDPOINT ÖZETİ (HAZIR)

İşlev	Method & Path	Not
TRTC UserSig	POST /api/trtc/token	trtcRoomId , numericUid dahil
Odaya gir (birleşik)	POST /api/live/join-room	password , otomatik koltuk, TRTC+state birlikte
Oda durumu	GET /api/chat/rooms/{room-Id}/state	11 koltuk + TRTC bilgisi
Presence/heartbeat	POST /api/chat/rooms/{room-Id}/presence	password , otomatik koltuk destekli
Koltuk işlemleri	POST /api/live/seats	take/leave/swap/force (0-10)
SSE akış	GET /api/chat/rooms/{room-Id}/stream	gift(render meta)/room_event/presence...
Sesli oda hediye	POST /api/chat/rooms/{roomId}/gifts	render meta döner
Birleşik hediye	POST /api/live/gift/send	voice/stream, render meta
Yayın hediye	POST /api/video-streams/{streamId}/gifts	giftRender döner
Son hediyeler / lider	GET /api/chat/rooms/{room-Id}/gifts	recentGifts(render meta) +leaderboard
Bildirimler	GET /api/notifications	data (JSON) derin bağlantı taşır

Tüm iş/finans mantığı backend'de; Flutter sadece bu verileri render eder ve kullanıcı aksiyonlarını bu endpoint'lere iletir.

9. CANLI ODA SİSTEMİ — KRİTİK HATA DÜZELTMELERİ (ZORUNLU)

Bu bölüm **canlı oda** (sesli oda + canlı yayın) sistemindeki kritik hataları giderir. Kural: **hiçbir şey lokal çalışmaz** — tüm hediye, koltuk ve senkron durumu **backend olaylarından** (SSE) sürülür. Flutter yalnızca render eder. Bu bölüm §3 ve §5 ile birlikte uygulanır.

9.1 Hediye senkronizasyonu — animasyon YALNIZCA backend olayıyla başlar

Backend akış sırası (değiştirilemez): jeton düş → GiftTransaction → GiftHistory → Room-GiftEvent → **SSE ile odadaki/yayındaki HERKESE** yayınla. Yani gönderen dahil herkes animasyonu **aynı anda** ve aynı SSE olayından görür.

Flutter kuralları (KESİN):

- Hediye butonuna basınca lokal animasyon BAŞLATMA.** Sadece `POST /api/live/gift/send` (veya ilgili gönderim endpoint'i) çağrılır. Animasyon, o çağrının HTTP yanıtından değil, **SSE type: 'gift' olayı** geldiğinde başlar. Böylece gönderenin cihazı da diğerleriyle **birebir aynı anda** oynatır.
- Dedupe (tekilleştirme):** Her olayın `giftId` alanı vardır. Gösterilen `giftId` 'leri bir `Set<String> _shownGiftIds` içinde tut; aynı `giftId` tekrar gelirse (reconnect replay vb.) **yeniden oynatma**.
- Geç bağlananlar eski hediye GÖRMEZ:** Odaya/yayına girerken `joinTimestamp = Date-Time.now().millisecondsSinceEpoch` sakla. Gelen `gift` olayında `event.timestamp < joinTimestamp` ise **animasyonu atla** (yalnızca gönderen paneli/sayaç için sessizce kullanılabilir, ama tam ekran/koltuk animasyonu oynatılmaz). Late-joiner asla eski hediye animasyonu görmez.
- GET .../gifts (recentGifts) ile ASLA animasyon başlatma.** O endpoint yalnızca "gönderenler paneli" ve lider tablosu içindir. Animasyonun **tek** tetikleyicisi SSE'dir.

```
final Set<String> _shownGiftIds = {};
late final int _joinTimestamp; // odaya girerken set edilir

void onSseGift(GiftEvent e) {
  // 1) katılımdan önceki hediyeler animasyon oynatmaz
  if (e.timestamp < _joinTimestamp) { _updateSenderPanelOnly(e); return; }
  // 2) tekilleştir
  if (!_shownGiftIds.add(e.giftId)) return; // zaten oynatıldı
  // 3) animasyonu backend metasına göre oynat (§9.2)
  _playGift(e);
  _updateSenderPanel(e); // §4 gönderen paneli + sayaç
}
```

9.2 Video/animasyonlu hediyeler — assetFormat 'a göre oynatıcı seç

Backend artık **kesin formatı** `assetFormat` ile gönderiyor. Oynatıcıyı **buna göre** seç (`assetType` değil, `assetFormat` esas):

assetFormat	Flutter oynatıcı
png / jpeg / webp / avif	CachedNetworkImage (imageUrl)
gif	Image.network veya CachedNetworkImage (imageUrl)
svga	svga / svgaplayer_flutter (imageUrl)
lottie	lottie paketi (imageUrl)
mp4 / webm	video_player (videoUrl) — preload + oynat + bitince dispose

Video kuralları (bellek sızıntısı yok):

```
Future<void> _playVideoGift(GiftEvent e) async {
  final controller = VideoPlayerController.networkUrl(Uri.parse(e.videoUrl!));
  await controller.initialize();           // preload
  controller.setLooping(false);
  await controller.play();
  controller.addListener(() async {
    if (controller.value.position >= controller.value.duration) {
      await controller.pause();
      await controller.dispose();          // bitince MUTLAKA dispose – bellekte tutma
      _removeOverlay(e.giftId);
    }
  });
}
```

- Aynı anda en fazla 1-2 tam ekran video overlay tut; kuyruk (queue) ile sırala.
- `thumbnailUrl` varsa video hazırlanana kadar poster olarak göster (siyah ekran olmasın).
- Tam ekran / koltuk-altı / küçük yerleşim kuralları **§3.3** ile aynıdır; yalnızca oynatıcı seçimi `asset-Format` 'a taşınır.

9.3 Jeton metni — sadece sayı

Bakiye/jeton gösteriminde **“Toplam” ve “Jeton” kelimelerini kaldır**, yalnızca sayıyı göster.

- `~~ Toplam 999 Jeton ~~` → `999` (istenirse yanında küçük 💎 ikon).
- Bu tüm ekranlarda geçerli (üst bar, cüzdan, hediye gönderim paneli). Sayı formatı: binlik ayraç uygulanabilir (`1.250`), ama kelime yok.

9.4 Odadan çıkış / yeniden girme (ghost koltuk hatası)

Hata: çıkıp tekrar girince koltuk boş görünüyor ama oturulamıyordu. **Backend düzeltmesi:** koltuk doluluğu artık **kısa “stale” penceresi** (`SEAT_STALE_MS = 45sn`) ile kontrol edilir. Yani bir kullanıcı 2 heartbeat (`>45sn`) kaçırırsa koltuğu **anında** boşa düşer. **Flutter bu yüzden düzenli heartbeat göndermeli ve çıkışta her şeyi temizlemeli.**

Heartbeat: odada iken her **~15 saniyede bir** `POST /api/chat/rooms/{roomId}/presence` (veya `join-room heartbeat`) çağır. (Backend eşiği `45sn = 3 heartbeat toleransı`.)

Odadan çıkışta ZORUNLU teardown sırası:

```
Future<void> leaveRoom() async {
  // 1) Backend'e bildir - koltuğu ve presence'ı bırak
  await api.post('/api/live/seats', {'action': 'leave', 'roomId':
roomId}); // koltuğu bırak
  await api.leaveRoom(roomId); // presence delete (?leave=1) → backend parti-
cipant_left yayınlar, 30sn beklemez
  // 2) TRTC
  await trtc.stopLocalAudio();
  await trtc.exitRoom();
  await trtc.dispose(); // TRTC instance dispose
  // 3) SSE
  await _sseSubscription?.cancel();
  _sseClient?.close();
  // 4) Zamanlayıcılar
  _heartbeatTimer?.cancel();
  _giftQueueTimer?.cancel();
  // 5) Controller / animasyon / video
  for (final c in _videoControllers) { await c.dispose(); }
  _videoControllers.clear();
  _lottieController?.dispose();
  _seatScrollController?.dispose();
  _messageFocusNode?.dispose();
  // 6) Cache temizle (§9.6)
  _clearAllRoomCaches();
}
```

- Çıkışta backend **beklemeden** `room_event: user_left` yayınlar; diğer istemciler koltuğu hemen boşaltır.
- Yeniden girişte `POST /api/live/join-room` → boş koltuk varsa **otomatik oturur** (§1.2), `POST /api/live/seats action:'take'` da çalışır. 45sn stale sayesinde eski hayalet kayıt engel olmaz.

9.5 Tek SSE bağlantısı + Last-Event-ID ile kesintisiz reconnect

- Oda başına **TEK** SSE bağlantısı aç (`GET /api/chat/rooms/{roomId}/stream`). Tüm olaylar (gift/seat/join/leave/mic/admin/mute/kick/music/pk/message/system/room_event) **bu tek akıştan** dinlenir. Ayrı ayrı bağlantı açma.
- **Last-Event-ID:** Backend her olay grubunda `id: <timestamp>` gönderir. Bağlantı koparsa, en son alınan id ile yeniden bağlan:
- Web `EventSource` bunu otomatik yapar.
- Flutter'da (özel SSE istemcisi) son `id` 'yi sakla ve reconnect'te **Last-Event-ID: <sonId>** header'ı (veya `?lastEventId=<sonId>` query) ile bağlan. Backend o zaman damgasından sonra kaçırılan olayları **tekrar oynatır** → hiçbir olay kaçmaz.
- Reconnect stratejisi: exponential backoff (1s, 2s, 4s... max ~15s). Yeniden bağlanınca ek olarak `GET .../state` ile koltuk/katılımcı haritasını tazele (kesin doğruluk için).

```
String? _lastEventId;
StreamSubscription? _sseSubscription;

void _connectSse() {
  final headers = {'Authorization': 'Bearer $jwt'};
  if (_lastEventId != null) headers['Last-Event-ID'] = _lastEventId!;
  _sseSubscription = sseClient.connect(streamUrl, headers: headers).listen(
    (frame) {
      if (frame.id != null) _lastEventId = frame.id; // sonra reconnect'te geri
      gönder
      _dispatchEvent(frame); // tek switch: type'a göre
    },
    onError: (_) => _scheduleReconnectWithBackoff(),
    onDone: () => _scheduleReconnectWithBackoff(),
  );
}
```

9.6 Bellek temizliği + cache yönetimi (sızıntı yok)

Odadan çıkışta / oda değiştirirken **her şeyi** temizle:

- **Controller'lar:** video, lottie, animation, scroll, text, seat controller → `dispose()` .
- **Stream/subscription:** SSE, mesaj stream, presence stream → `cancel()` .
- **Timer'lar:** heartbeat, gift queue, animasyon süreleri → `cancel()` .
- **Focus/audio/rtc/player:** focus node dispose, audio session stop, TRTC dispose, video player dispose.
- **Cache'ler (tümü):** RoomCache , GiftCache , SeatCache , ParticipantCache , MessageCache , AnimationCache → temizle. Ayrıca `_shownGiftIds.clear()` .

```
void _clearAllRoomCaches() {
  roomCache.clear(); giftCache.clear(); seatCache.clear();
  participantCache.clear(); messageCache.clear(); animationCache.clear();
  _shownGiftIds.clear(); _recentSenders.clear();
}
```

- **Görüntü cache limiti:** uygulama başında `PaintingBinding.instance.imageCache.maximumSizeBytes = 100 << 20; // 100MB` . Oda değişiminde büyük asset'ler için `imageCache.clear()` / `clearLiveImages()` çağrılabilir.

9.7 Performans (donma/kasma yok)

- **State yönetimi:** Riverpod veya Bloc kullan; `setState` ile tüm ekranı yeniden çizme. Koltuk, mesaj, hediye katmanlarını ayrı provider/bloc ile böl.
- **const widget'lar:** değişmeyen widget'ları `const` yap.
- **AutomaticKeepAlive** : yalnızca gerçekten gereken (ör. sekme içeriği) yerlerde kullan; her yerde değil.
- **RepaintBoundary** : hediye animasyon overlay'i, koltuk grid'i ve mesaj listesi gibi sık çizilen katmanları `RepaintBoundary` ile sar → gereksiz repaint yayılmasın.
- **Sanal liste (virtual scroll):** mesaj/katılımcı/hediye listelerinde `ListView.builder` (lazy). Tümünü belleğe alma.
- **Lazy loading:** ağır asset'leri (video/lottie) yalnızca gösterileceği anda yükle; önizleme için `thumbnailUrl` .

9.8 Görüntü performansı

- **Formatlar:** PNG/WebP/AVIF desteği (`assetFormat` 'a göre). Mümkünse WebP/AVIF tercih et (daha küçük).
- **Thumbnail:** liste ve poster için `thumbnailUrl` kullan; tam çözünürlüğü yalnızca gerektiğinde.
- **CachedNetworkImage :** tüm ağ görselleri için; `placeholder` = `thumbnailUrl` veya shimmer, `error-Widget` = ikon/emoji fallback.
- **Precache:** odaya girerken sık kullanılan koltuk/hediye ikonlarını `precacheImage` ile önceden yükle → ilk gösterimde takılma olmaz.

9.9 Sonuç (bu bölümün kabul kriterleri)

- [] Hediye animasyonu **yalnızca SSE gift olayıyla** başlar; gönderen dahil **tüm cihazlarda aynı anda** oynar; lokal başlatma yok.
- [] Aynı `giftId` iki kez oynatılmaz (dedupe); katılımdan önceki hediyeler (`timestamp < join-Timestamp`) animasyon oynatmaz.
- [] Video hediyeler (`mp4 / webm`) oynar; bitince controller **dispose** edilir; bellek şişmez. `svg / lot-tie / gif / png` doğru oynatıcıyla gösterilir (`assetFormat`).
- [] Jeton metninde “Toplam” ve “Jeton” **yok**; sadece sayı görünür.
- [] Odadan çıkıp tekrar girince koltuğa **oturulabiliyor** (45sn stale + tam teardown); koltuklar gerçek zamanlı doğru.
- [] Oda başına **tek** SSE bağlantısı; kopunca **Last-Event-ID** ile kaçırılan olaylar tekrar alınır.
- [] Odadan çıkışta tüm controller/stream/timer/cache **temizlenir**; bellek sızıntısı yok.
- [] Uygulama akıcı (Riverpod/Bloc + const + RepaintBoundary + sanal liste + lazy load); donma yok.